多執行緒演算法

動態多執行緒以依賴 DAG 分開工作總量、關鍵路徑與排程。本章從 spawn、sync 和競爭修復出發，推導矩陣乘法與平行 merge/sort 的演算法、證明和空間條件。

先備知識為 DAG、遞迴分析與並行成本模型；我們也會區分平行度上界與增加處理器後的實際收益。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態多執行緒基礎

動態多執行緒模型讓程式描述哪些計算可以並行，由排程器決定何時、在哪個處理器執行。邏輯task不是固定綁定的OS thread；spawn不承諾立刻取得另一個核心。

本章採相同速度的P個理想處理器與循序一致的共享記憶體：每條指令可看成某個符合各依賴的交錯順序，load/store各自是單一步驟；讀、加、寫的合成操作不自動具有原子性。單位成本算術、索引、spawn/sync為常數工作，排程器本身的時間、頻寬、cache及通訊成本不計入此理想界。

非空有限計算用DAG表示。節點可是一個strand（不含平行控制的一段順序指令），帶自己的正執行時間；空strand可刪除，要用單位節點證明時，可再拆成單位指令鏈。控制與資料依賴決定有向邊，而不是把畫面上同層的節點一律當成可並行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Spawn、call、sync 的確切語義

`x = spawn F(...)`允許F與呼叫者後續程式並行；普通`y = G(...)`則呼叫者須等待G返回。`sync`等待本程序此前spawn的child全部完成，才執行後面的指令；它不是全系統的barrier。

spawn參數先在parent求值。child的區域變數屬該次呼叫，指向共享陣列的參數仍可能讓多task存取同一格；不能只看變數名稱不同就判獨立。讀spawn回傳值之前必須有相應依賴，通常明寫sync。

本章程序返回前隱含sync，但若先求`return x+y`的值才返回，隱含等待不足以允許提早讀x；因此先sync再算回傳式。刪掉spawn/sync並將parallel for改普通for得到serialization；它能解出正確答案，不代表所有平行交錯也正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci 範例的程式與正確性

```
P-FIB(n):                 // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

兩個child只讀自己的n並回傳各自結果，沒有共享寫入。n下降使呼叫終止；基例F0=0、F1=1，加上歸納假設與sync後的x+y，得到正確Fn。若把兩個child都spawn，仍必等兩者後才能相加，漸近work/span不變，只改常數。

這個版本故意沒有memoization，同一參數會算多次。高平行度不等於work有效率：順序迭代法只需O(n)次算術，不能因遞迴版本可以占用更多核心，就忽略其指數工作量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一 strand 模型的 work 與 span

為了逐點驗算，將每個非基例呼叫分為三個單位strand：pre（到spawn）、cont（到普通第二呼叫）、join（sync後相加返回）；每個基例為一個單位strand。pre先於左child及cont，cont先於右child，兩個child結束都先於join。

令W為全部strand成本，D為最長依賴路徑成本，則

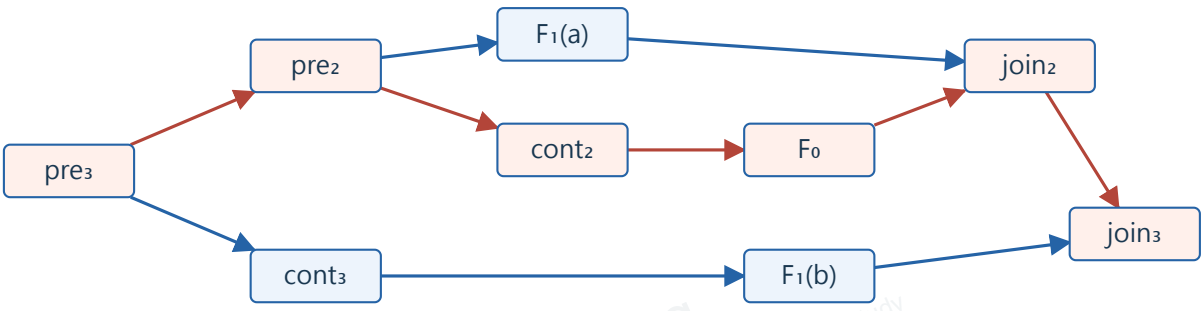
$$W_n = W_{n-1} + W_{n-2} + 3, \quad W_0 = W_1 = 1;$$
$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \quad D_0 = D_1 = 1.$$

n	Fn	Wn	Dn
2	1	5	4
3	2	9	6
4	3	17	8

Wn=4F(n+1)−3可直接代入歸納，Dn=2n對n≥2成立。令 $\varphi = (1+\sqrt{5})/2$；由φ²=φ+1，歸納得φ^(n−2)≤Fn≤φ^(n−1)（n≥2）：n=2、3可核對，兩界的相鄰和仍是下一個φ次方。因此W=Θ(φ^n)、D=Θ(n)。這是明列的單位strand模型，不把一個完整遞迴呼叫誤算成一個O(1)節點。

九個 strand 的依賴可逐條追蹤

P-FIB(3) 的完整九個 strand：箭頭是先後依賴



每格恰為一個單位 strand；W=9。紅色最長路徑含6格，因此 D=6。

圖的pre3先分到pre2及cont3；cont3才呼叫右側F1。pre2也先分到其F1與cont2，cont2再呼叫F0。兩個基例都完成才可做join2，join2與右側F1都完成才可做join3。

紅路徑pre3→pre2→cont2→F0→join2→join3共6個節點，W=9、D=6。另一條pre3→cont3→F1→join3只有4個節點。圖上箭頭是依賴方向，不是把同一呼叫的所有節點排成一條長鏈。

Work law、span law 與可達的加速比

記 $W = T_1$、$D = T_\infty$、實際P處理器時間 T_P。每個時間單位最多完成P單位工作，且關鍵路徑不可並行，故

$$T_P \geq \max\{W/P, D\};$$

單位節點離散模型還有 $T_P \geq \lceil W/P \rceil$。加速比 $S_P = W/T_P \leq \min(P, W/D)$，效率 $S_P/P \leq 1$。平均平行度A=W/D限制加速比；P>A便無法有完美的P倍加速，不代表第P+1顆處理器必然沒有收益。

順序合成兩段時，W相加、D也相加；若兩段獨立地fork後join，W仍相加，D取兩段最大值，另加明列的fork/join成本。空計算W=D=0直接時間0，不把0/0稱為平行度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy 排程的完整上界證明

每個時間步，只要有至少P個ready節點便執行P個；不足P時全部執行，這是greedy。令完整步C、不完整步I，總時間T=C+I。計算尚未結束時，每個不完整步至少執行一個節點，故W≥PC+I。

不完整步會移除剩餘DAG的所有source。步後的最長路徑，可在該步開始前的DAG中向前延伸到至少一個被移除的source；若步後已空，步前span也至少1。因此每個不完整步使剩餘span至少減1，最多有D個不完整步，I≤D。代入得

$$T \leq \frac{W - I}{P} + I \leq \frac{W - D}{P} + D \leq W/P + D.$$

最後一步使用P≥1使I的係數1−1/P非負。單位節點證明可對整數長度strand拆鏈後套用；它不是無成本處理任意不可切割、長度未知task的另一個模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩倍近似與 slackness 的量化

最優排程也受 $T_P^* \geq \max(W/P, D)$ 限制，故greedy的T≤W/P+D≤2T*。更精確地可得 $(2 - 1/P)T_P^*$ 上界，但無需預知未來DAG就有常數比保證。

定義slackness $\sigma = W/(PD) = A/P$。對D>0，上下界給

$$1 \leq \frac{T_P}{W/P} \leq 1 + \frac{1 - 1/P}{\sigma} \leq 1 + 1/\sigma.$$

所以σ很大時，理想greedy時間接近W/P；這是數學界，不是對真機頻寬或排程器開銷的實測。最優排程可以停用多餘處理器，所以其時間隨P不增；任意實作的測得時間另受開銷與排程選擇影響。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平行度30，第31顆仍有收益

697個互相獨立的單位工作全部完成後，才能開始23個單位工作的依賴鏈。W=697+23=720，D=1+23=24，A=30。

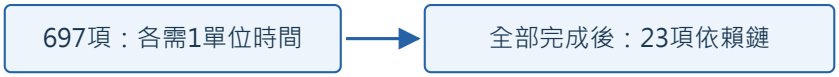
P	獨立階段的完整配置	階段時間	鏈時間	精確最短時間
30	23輪各30項，再7項	24	23	47
31	22輪各31項，再15項	23	23	46

所有前驅都完成前鏈不能開始，所以ceil(697/P)+23同時是可行排程與下界。
S30=720/47≈15.32、S31=720/46≈15.65，均≤30，但增加處理器確實仍有收益。P≥697才可把第一階段縮成1輪、達到D=24。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

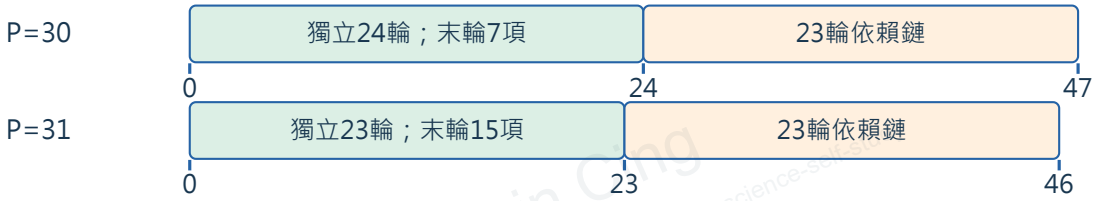
平行度30，第31顆仍有收益 （續）

相同 DAG：31個處理器仍比30個快



W=697+23=720
D=1+23=24

平均平行度 W/D=30；以下兩列使用相同時間比例尺。



$T_{30}=47$、$T_{31}=46$：加速比至多30，不代表第31個處理器沒有收益。

減 work 但增加 span，可能改變優劣

兩個正確版本分別有(W,D)=(1024,4)、(768,16)，都用理想greedy排程。

P=8時，第一版時間至少1024/8=128，第二版時間至多768/8+16=112，因此第二版一定較快。
P=128時，第一版至多1024/128+4=12，第二版至少16，因此第一版反而一定較快。

這個判定用到不重疊的上下界，不把W/P+D的上界誤當精確時間。若兩版本的可行時間區間重疊，單靠W,D未必能判定哪個實際較快。少量核心上的單次時間，也不足以識別work與span各自的改變。

Parallel for 如何以 fork/join 實作

以下與合併算法均採0-based半開區間[l,r)，空區間l=r。

```
PAR-FOR(l,r,body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l,m,body)
    PAR-FOR(m,r,body)
    sync
```

n個非空迭代恰對應n葉、n−1內節點；每個內節點常數spawn/sync工作，所以額外work O(n)，不會憑空多出n lg n。最長呼叫與join路徑深度Θ(lg n)，因此總span是Θ(lg n)+各迭代span最大值，前提是各迭代真的獨立。

若用普通for依序spawn n個task，parent本身就有Θ(n)長的發出鏈，不能宣稱同樣對數span。平行迴圈結尾的sync也確保下一段讀取所有結果時已完成。

Matrix-vector：只平行不同 row 是安全的

```
MAT-VEC(A,x):
    n = A.rows
    allocate y[n]
    parallel for i = 0 to n-1:
        y[i] = 0
        for j = 0 to n-1:
            y[i] += A[i,j]*x[j]
    return y
```

每個i獨占y[i]，A,x只讀；內圈第j步後是前j+1個乘積的和，所以正確。work $\Theta(n^2)$，span Θ(lg n)+Θ(n)=Θ(n)，平均平行度Θ(n)。n=0回空向量。

不能只把內圈也改parallel for：各j對同一y[i]做讀改寫，會產生determinacy race（未由依賴排序的衝突存取）。兩個邏輯並行存取同格、至少一個寫入便有此衝突；即使碰巧寫相同值，也不能用「跑過一次正確」證明獨立。

循序一致仍會遺失更新

遺失更新：逐指令讀值、暫存值與最後寫回

初始 x=0；每個 task 內部仍依 load → add → store。

時間步	Task 1	Task 2	共享 x
1	load $r_1 \leftarrow x = 0$		0
2		load $r_2 \leftarrow x = 0$	0
3	add $r_1 \leftarrow r_1+1 = 1$		0
4	store $x \leftarrow r_1$		1
5		add $r_2 \leftarrow r_2+1 = 1$	1
6		store $x \leftarrow r_2$	1

兩者結尾 sync 後讀到1：等待完成不能把三條指令變成不可分割操作。

圖的交錯尊重各task的load→add→store，最後x卻為1；如果第一個task全部完成才換第二個，會得到2。循序一致只限制單條指令的交錯，不把三條指令合成原子操作。

本章以互不衝突的分工或reduction消除此類競爭。原子加或鎖也可建立依賴，但其同格序列化、工作量與span必須另算；不能保留「全部加法同時完成」的分析。

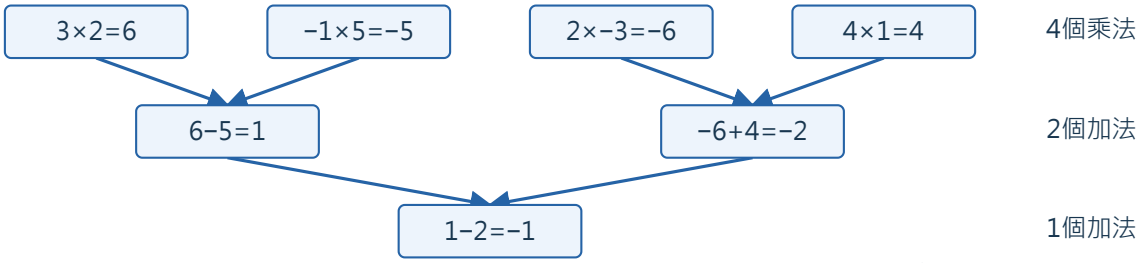
用 reduction 正確地平行內積

```
DOT(Arow,x,l,r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m = floor((l+r)/2)
    left = spawn DOT(Arow,x,l,m)
    right = DOT(Arow,x,m,r)
    sync
    return left+right
```

每個child只回傳自己的值，沒有共同累加格；結合律與歸納保證結果是區間內積。work為$\Theta(n)$，span $\Theta(\lg(n+1))$。把MAT-VEC每個row改成這個DOT，得到$\Theta(n^2)$work、$\Theta(\lg n)$span（$n \geq 2$）。例row=[3,−1,2,4]、x=[2,5,−3,1]，乘積[6,−5,−6,4]，第一層和[1,−2]，最後−1。

用 reduction 正確地平行內積 （續）

內積不是共寫一個總和：四個私有乘積沿樹合併



算術 DAG：work=7、span=3；fork/join 另有 Θ(log n) 控制深度。

對浮點加法，固定的樹可能和順序累加有不同捨入結果；正確性證明先在精確、可結合算術下成立，不能順手承諾逐bit相同。

粒度與記憶體配置的模型

將n個常數成本迭代分成每批最多g個、以平衡PAR-FOR發出各批，1≤g≤n，work Θ(n)，span Θ(lg⌈n/g⌉+g)。較大g減少task常數開銷，也增加每葉序列成本；是否值得依P及slackness而定。

本章allocate取得未初始化、互不重疊儲存空間的描述符，抽象為常數span；若實作需要逐格初始化，必須計入相應工作並以parallel loop初始化才保留對數span。不是把一個串行清零n^2格的操作藏進O(1)。

矩陣區塊用(row起點、column起點、stride、階數)的view表示，不隱藏整塊複製。任務控制區、分配器、cache與記憶體頻寬在實作時另有成本；以下數學界皆在已明列的理想模型內。

多執行緒矩陣乘法

```
P-MULTIPLY(A,B):
    n = A.rows                  // both n by n
    allocate C[n,n]
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1:
            C[i,j] = 0
            for k = 0 to n-1:
                C[i,j] += A[i,k]*B[k,j]
    return C
```

每個(i,j)獨占一格輸出，內圈仍依序累加。work $\Theta(n^3)$，span兩層發出樹$\Theta(\lg n)+\Theta(\lg n)$，再加一條內圈$\Theta(n)$，總$\Theta(n)$，平均平行度$\Theta(n^2)$。兩個parallel關鍵字沒有消除每格的n項依賴。

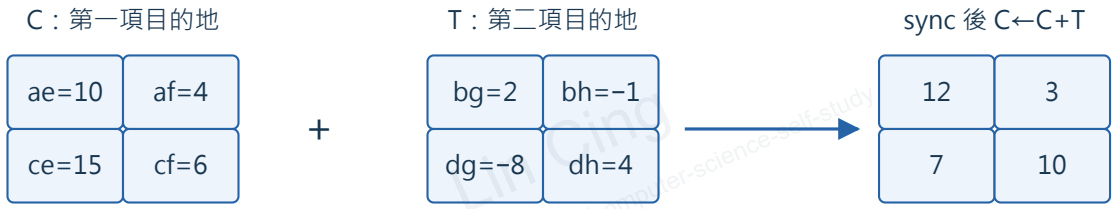
若每個Cij改由前述DOT類型的二分內積回傳，work仍$\Theta(n^3)$，span則$\Theta(\lg n)$，平均平行度$\Theta(n^3/\lg n)$。直接並行寫入Cij不是這個reduction算法。

八次子乘法必須寫八個不同目的地

令A=[[a,b],[c,d]]、B=[[e,f],[g,h]]，此處字母是同階區塊。先計C=[[ae,af],[ce,cf]]與暫存T=[[bg,bh],[dg,dh]]，八個乘積各有獨占目的地。sync後才做C=C+T。

數例A=[[2,−1],[3,4]]、B=[[5,2],[−2,1]]：C暫時[[10,4],[15,6]]，T=[[2,−1],[−8,4]]，相加得[[12,3],[7,10]]。

八個獨立乘法：目的地分開，回收時才相加



八個乘法各寫 C 或 T 的一格；先等待全部乘法，再逐格相加。

A=[[2,−1],[3,4]]，B=[[5,2],[−2,1]]；同格的兩項沒有同時做 +=。

若把ae與bg兩個task都寫成C11+=product，兩者便衝突。分出T是在用額外空間解除寫入依賴，不是只加spawn就能平行。

遞迴矩陣乘法的完整程序

```
P-REC(C,A,B):
    n = A.rows
    if n == 1: C[0,0] = A[0,0]*B[0,0]; return
    allocate T[n,n]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    view C and T into four n/2 by n/2 blocks
    spawn P-REC(C11,a,e); spawn P-REC(C12,a,f)
    spawn P-REC(C21,c,e); spawn P-REC(C22,c,f)
    spawn P-REC(T11,b,g); spawn P-REC(T12,b,h)
    spawn P-REC(T21,d,g); P-REC(T22,d,h)
    sync
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1: C[i,j] += T[i,j]
```

前提為同階方陣、階數是2的冪，A與B只讀，C不與它們重疊。任意正階數先補零到下一個2冪再取回結果，n=0直接回空矩陣。T及每個child工作區也獨立；八個spawn/call數量是固定常數，不是n長的發出鏈。

八路矩陣乘法的歸納與遞迴成本

基例寫出正確單乘積。歸納假設各n/2子乘積正確，八個child因目的地不同而無競爭；sync確保全部乘積可讀，再逐格相加，依區塊乘法恆等式得到AB。只有共同讀A、B不構成衝突。

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\lg n) = \Theta(\lg^2 n).$$

span沿八個child中一條最長路徑，不是八者相加；每層回收的nested parallel for又有Θ(lg n)發出／join深度，不能當O(1)。n=2^L時Σr=1..L r=Θ(L^2)，同時給上、下界。n=4有8^2=64個純量乘法葉；平行化沒有少做乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 平行化的三個同步階段

本章以 M_i 表示第 13 章的 P_i，兩者是同一個乘積。先以平行逐格加減準備(a+d)、(e+h)、(c+d)、(f−h)、(g−e)、(a+b)、(c−a)、(e+f)、(b−d)、(g+h)。輸入準備完成後，七乘積同時可執行：

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

七者完成後sync，再平行回收C11=M1+M4−M5+M7，C12=M3+M5，C21=M2+M4，C22=M1−M2+M3+M6。不能在輸入和差未完成前spawn消費它們，也不能在乘積未完成前回收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 的 work、span 與平行工作區

每層七個同階child，常數個矩陣加減：

$$\begin{aligned} W_S(n) &= 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \\ D_S(n) &= D_S(n/2) + \Theta(\lg n) = \Theta(\lg^2 n). \end{aligned}$$

正確性由第 13 章已展開的四個區塊恆等式及遞迴歸納，平行化另需上述三階段依賴。其平均平行度Θ(n^(lg7)/lg²n)小於八乘法版本，卻具有較低work；平行度的比值不是單獨選算法的目標。

順序深度優先的O(n²)暫存不能直接移給所有child同時展開的版本。若每個呼叫分配自己的Θ(n²)工作區並保持到join，八路的完全展開空間滿足Q(n)=8Q(n/2)+Θ(n²)=Θ(n³)；七路則為Θ(n^(lg7))。有限P與工作區重用可降低峰值，但必同時核對排程與span，不能沿用一個未實作的空間界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平行 Strassen 的可執行規格

定義PADD(X,Y,s)配置獨立R，以兩層parallel for寫R[i,j]=X[i,j]+s·Y[i,j]，等待迴圈完成再回R。取s=±1；其work $\Theta(n^2)$、span $\Theta(\lg n)$。以下A、B同階且先補到2冪，n=0由外層直接回空矩陣。

```
P-STRASSEN(A,B):
    n = A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    S1 = PADD(a,d,+1); S2 = PADD(e,h,+1)
    S3 = PADD(c,d,+1); S4 = PADD(f,h,-1)
    S5 = PADD(g,e,-1); S6 = PADD(a,b,+1)
    S7 = PADD(c,a,-1); S8 = PADD(e,f,+1)
    S9 = PADD(b,d,-1); S10 = PADD(g,h,+1)
    M1 = spawn P-STRASSEN(S1,S2)
    M2 = spawn P-STRASSEN(S3,e)
    M3 = spawn P-STRASSEN(a,S4)
    M4 = spawn P-STRASSEN(d,S5)
```


平行 Strassen 的可執行規格 （續）

```
    M5 = spawn P-STRASSEN(S6,h)
    M6 = spawn P-STRASSEN(S7,S8)
    M7 = P-STRASSEN(S9,S10)
    sync
    allocate C[n,n], viewed as four blocks
    parallel for i = 0 to n/2-1:
        parallel for j = 0 to n/2-1:
            C11[i,j] = M1[i,j]+M4[i,j]-M5[i,j]+M7[i,j]
            C12[i,j] = M3[i,j]+M5[i,j]
            C21[i,j] = M2[i,j]+M4[i,j]
            C22[i,j] = M1[i,j]-M2[i,j]+M3[i,j]+M6[i,j]
    return C
```

十個PADD在此依序呼叫，數量固定，故總span仍$\Theta(\lg n)$；先全部完成才spawn七乘法，與三階段依賴一致。沒有把任意數量的串行發出忽略掉。

多執行緒合併排序

只把兩個遞迴sort並行、仍用順序MERGE，並不充分：

```
PAR-SORT-SERIAL-MERGE(A,l,r):
    if r-l <= 1: return
    m = floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A,l,m)
    PAR-SORT-SERIAL-MERGE(A,m,r)
    sync
    serial MERGE the two sorted halves using an auxiliary buffer
```

work為2W(n/2)+Θ(n)=Θ(n lg n)，span卻是D(n/2)+Θ(n)=Θ(n)，平均平行度僅Θ(lg n)。合併前的sync必要，否則會讀到未排序完成的run；要進一步縮短span，必須平行化merge本身。

二分切點先定義半開區間與平手規則

兩個輸入run位於只讀陣列T的[l1,r1)、[l2,r2)，輸出O[o,o+N)不得覆寫T，N為兩段長度和。為保證穩定性，每個record以(key,原始位置)作總序比較；原始位置唯一，相同key仍按原先順序。

```
LOWER-BOUND(T,l,r,x):
    while l < r:
        m = floor((l+r)/2)
        if T[m] < x: l = m+1
        else: r = m
    return l
```

不變量是原區間中小於x者的切點位於當前[l,r]；在原輸入run內，當前l左方都<x、r右方（含r）都≥x。每次依中點比較保留可能切點，長度嚴格縮小；空run直接回l，永遠不讀不存在的T[l]。工作及span O(1+lg(n2+1))，明列空run的常數處理。

可處理空 run 的完整 Parallel Merge

```
P-MERGE(T,l1,r1,l2,r2,O,o):
    n1 = r1-l1; n2 = r2-l2
    if n1 < n2: swap (l1,r1) with (l2,r2); swap n1,n2
    if n1 == 0: return
    m = floor((l1+r1)/2)
    j = LOWER-BOUND(T,l2,r2,T[m])
    q = o+(m-l1)+(j-l2)
    O[q] = T[m]
    spawn P-MERGE(T,l1,m,l2,j,O,o)
    P-MERGE(T,m+1,r1,j,r2,O,q+1)
    sync
```

先把較長run放第一描述符；交換的是view，不移動或重新標記record。若第一段空，第二段也空；若僅第二段空，仍從第一段取median逐步定位，不讀空段。每次拿走一個pivot，兩個child總長N−1，因此有限輸入必終止。

輸出位置、正確性與穩定性的一般證明

左側應有m−l1個第一run元素、j−l2個第二run元素，總長q−o。它們都在總序上小於pivot；其餘元素都大於pivot。故pivot的全域輸出位置是q，兩個child輸出區間恰為[o,q)與[q+1,o+N)，彼此及pivot均不重疊。

對N歸納：N=0空合併正確；兩child均較小，由歸納假設正確合併相應元素。左答案、pivot、右答案串接已排序且每個輸入record恰出現一次，得到完整merge。T只讀、O的寫入區間互斥，所以沒有determinacy race，sync後全部結果可用。

相同key按全域原始位置比較，輸出保持原先相對次序，因此穩定。若只比key且一律使用同一lower_bound規則，長度交換可能把相等的右run元素放到左run之前；普通排序正確不自動等於穩定。

八個 record 的根切點與二分步驟

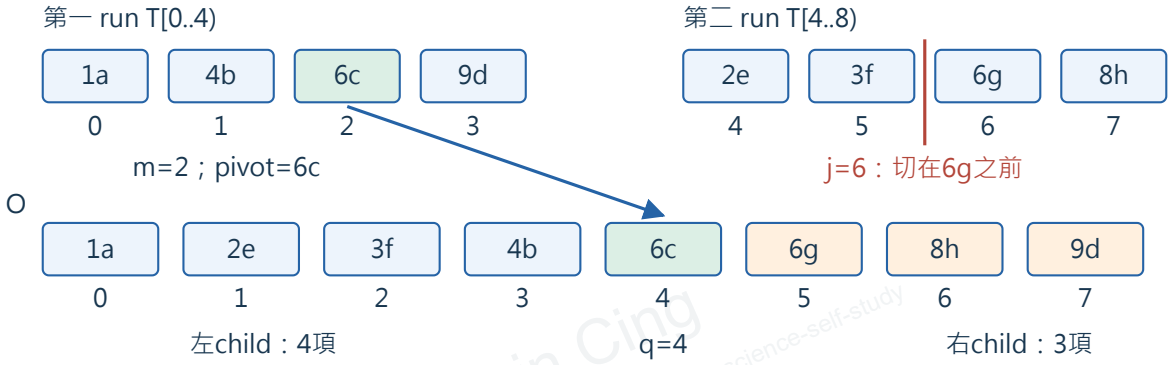
T=[1a,4b,6c,9d | 2e,3f,6g,8h]，字母依a..h表示原始位置0..7；比較先key再原始位置。l1=0,r1=4,l2=4,r2=8,o=0。第一run長4，m=2、pivot=6c。

當前搜尋[l,r)	mid	T[mid]與6c比較	更新
[4,8)	6	6g>6c（key相等、位置較後）	r=6
[4,6)	5	3f<6c	l=6
[6,6)	—	空搜尋區間	回j=6

q=0+(2−0)+(6−4)=4；左child共4元素，右child共3元素。

八個 record 的根切點與二分步驟 （續）

中位 pivot 的秩：$q=o+(m-l_1)+(j-l_2)=4$



比較 (key, 原始位置)：6c < 6g；左區、pivot、右區的輸出互不重疊。

同鍵案例：每個位置從哪裡來？

`A=[1a,4b,4c,8d]`，`B=[2e,4f,5g,9h]`；字母代表原始位置且 b < c < f。選 `x=4c`，A 左側長度 2；B 中第一個不小於 `4c` 的位置是 1，故輸出位置 `q=2+1=3`。

左任務合併 `[1a,4b]` 與 `[2e]`，右任務合併 `[8d]` 與 `[4f,5g,9h]`。兩側完成後得到 `[1a,2e,4b,4c,4f,5g,8d,9h]`。

中位數分割及目的位置

A　1a　4b | 4c | 8d　　pivot 4c → output[3]

B　2e | 4f　5g　9h　　B 的 lower_bound = 1

左任務: A[1a,4b] + B[2e]
→ [1a,2e,4b]

右任務: A[8d] + B[4f,5g,9h]
→ [4f,5g,8d,9h]

左區 [0,3)　pivot [3]　右區 [4,8)

同一數例的全部輸出寫入

以下為一個合法深度優先serialization的pivot寫入順序；平行執行可以改交錯，但目的格相同、各寫一次。

寫入序	pivot	q（0-based）	來源／輸出意義
1	6c	4	根的左右分界
2	4b	3	根左段的最後元素
3	3f	2	較長run交換後的median
4	2e	1	小run的切點
5	1a	0	另一run已空
6	8h	6	根右段交換後的median
7	6g	5	右段左child
8	9d	7	右段右child

最終O=[1a,2e,3f,4b,6c,6g,8h,9d]，兩個6仍c在g之前。即使改成先跑右child，只讀T及唯一輸出格也保證相同答案；不是靠整個陣列寫入順序維持排序。

Merge 最壞 span 的上界與下界

較長run長n1≥n2，取median後任一child至多⌊n1/2⌋+n2≤(N+n2)/2≤3N/4。即使一側切點落在端點，這個上界仍成立。二分搜尋至多O(lg(N+1))，所以

$$D_M(N) \le D_M(3N/4) + O(\lg(N+1)) = O(\lg^2(N+1)).$$

沿路每次固定比例縮小，有O(lg N)層，第r層剩餘log大小O(lg N−cr)，等差和給平方對數。

下界用近等長交錯run，例如[0,2,⋯,2m−2]與[1,3,⋯,2m−1]，m是2冪。左child持續成為同型的兩個等長交錯run，長度每層減半，且搜尋另一run的median切點仍需Θ(lg m)比較。一路累加lg m+(lg m−1)+⋯給Ω(lg²m)，所以最壞span為Θ(lg²N)。若另一run始終空，搜尋只需常數、span則Θ(lg N)；不能把最壞界稱為每個輸入的精確成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性 work：一次寫出不等於已付清搜尋

每個非空呼叫挑走一個不同pivot，所以有N個非空呼叫；各生兩child，含空返回共2N+1個呼叫。N次輸出給Ω(N)，但O(N)上界還必須支付每個非空節點的二分搜尋，不能只說「每元素寫一次」。

大小m的節點非遞迴工作≤c lg(m+1)。把此成本平均分攤給它仍含有的m個輸入record，每個收 c・lg(m+1)/m。固定record x只在一條ancestor鏈上被收費，直到自己被選成pivot為止。

從最後一次含x的呼叫向根反數r層，因child≤3parent/4，該層大小至少(4/3)^r。下一頁將此一般不等式求和，連空child也可處理，不需要取lg0或聲稱小規模的兩個child都至少N/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每個 record 的搜尋費總和有常數界

$g(u)=\lg(u+1)/u$ 在u>0遞減，因導數分子的符號等於u/(u+1)−ln(u+1)<0；後者由積分 ∫ 0^u 1/(1+t)dt>u/(1+u)得到。因此反數r層的分攤費至多

$$c\frac{\lg((4/3)^r+1)}{(4/3)^r}\le c\,[1+r\lg(4/3)](3/4)^r.$$

幾何和Σq^r=1/(1−q)，以及由有限和求導／取極限所得Σr q^r=q/(1−q)²，對q=3/4都收斂。因此每個record總費O(1)，N個record共O(N)；再加N+1個空呼叫的常數費，總work O(N)。與輸出下界合併，W_M(N)=Θ(N)。

這是對任意合法切點的一般計費證明；空run與不平衡的小child已包含，不以有限例子取代上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩個 buffer 的完整 Parallel Merge Sort

先將輸入record附上原始位置，令X與Y兩個獨立長n陣列都含相同輸入（一次parallel copy），呼叫SORT(X,Y,0,n)，最後取Y。

```
SORT(src,dst,l,r):
    if l == r: return
    if r-l == 1: dst[l] = src[l]; return
    m = floor((l+r)/2)
    spawn SORT(dst,src,l,m)
    SORT(dst,src,m,r)
    sync
    P-MERGE(src,l,m,m,r,dst,l)
```

每個子呼叫交換buffer角色；parent的兩個已排序run會留在src，sync後merge到dst。P-MERGE輸入與輸出必須是不同實體buffer。這個雙buffer寫法避免每個sort呼叫再配置長度n的暫存，不把原地寫入與讀取尚未消費的輸入混用。

配置成本與實務取捨

上界假設未初始化的緩衝區能以 O(1) 配置，並計入每次實際讀寫。如果語言的配置動作必須序列初始化 n 個元素，單次初始化就可能引入 Θ(n) span，不能照搬前述界。

簡單版本各層同時可能存在 Θ(n) 個緩衝位置，最多 O(n log n) 額外空間；也可預先配置兩個長度 n 的緩衝陣列，在遞迴層間交換來源與目的，達 O(n) 額外空間，但必須另證各層讀寫無衝突。

小問題可改用序列 merge/sort 作為 cutoff，以降低工作排程開銷。這會改變常數及 span，需要按 cutoff 重新分析；理論上界不等於實測加速。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Buffer 角色交換的正確性與競爭檢查

每個SORT進入時，兩個buffer的當前區間都仍含該區間的原始輸入。根由copy建立；parent在啟動兩child前未改自身區間，兩個child又只碰各自互斥的半區，故每個child進入時也成立。

基例把唯一record複製到dst。由歸納假設，兩child結束後，parent的src左右兩run分別排序且元素不變；sync後的P-MERGE把兩者穩定合併到dst。歸納完成排序規格；parent無需承諾src最後仍與dst相同，只需在呼叫入口滿足條件。

兩child即使交換buffer角色，也仍在相同的一對全域陣列中寫互斥索引區間；parent只在sync後合併。總資料workspace O(n)，峰值task描述符O(n)、每條呼叫鏈O(lg n)；初始化copy工作Θ(n)、span Θ(lg n)，不改後述排序階數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

排序的緊界不能只把最壞成本相加

每層merge work Θ(n)，有Θ(lg n)層，故W_S(n)=Θ(n lg n)。span上界是

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\lg^2 n) = O(\lg^3 n).$$

要證最壞下界，需同一輸入在沿途各層都產生昂貴merge。對n=2^L，按L位元反轉索引的順序放入鍵；n=8例[0,4,2,6,1,5,3,7]。根的兩半排序後是偶數與奇數交錯run；每個子問題的相對秩亦是相同構造。故每層都有前述Ω(lg²(size))的merge span，且parent merge在兩child都完成後才開始。

沿一條child鏈相加Σr=1..L r²=Θ(L³)，得到二冪規模的最壞D_S(n)=Θ(lg³n)。平均平行度為Θ(n/lg²n)，不是把merge的平方對數錯當sort的span。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非二冪規模也能構造同時昂貴的各層

任意n可遞迴安排各區間的相對秩：n偶數時，前半放偶秩、後半放奇秩；n奇數時，較短的前半放奇秩、較長的後半放偶秩。再對兩半各自的有序鍵集合做相同安排。每個sort節點排序後的兩run都互相交錯，長度差至多1；二冪時就是前頁的位元反轉族。

對這種merge，較長run的median左方有m個本run元素，另一run左方有m或m+1個。左child仍是長度差至多1的交錯run，總長介於N/2−1與N/2+1；N≥8時，兩run的長度也都為Θ(N)。所以沿該路徑有Θ(lg N)層，每層二分搜尋須Θ(lg N)比較（規模隨層數固定比例下降），總span為Θ(lg²N)。

同一遞迴秩配置讓每個sort節點都滿足此條件。沿較大的child走，規模每層約減半，且須等它完成後才開始parent merge，故可串接各層關鍵路徑，得到Ω(lg³n)。這補足任意n的緊下界，不只在不同層各自選互不相容的最壞輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一模型下比較 work 與最壞 span

算法	Work	最壞 span	平均平行度
遞迴P-FIB	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(\varphi^n/n)$
row平行matrix-vector	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
reduction matrix-vector	$\Theta(n^2)$	$\Theta(\lg n)$	$\Theta(n^2/\lg n)$
雙外圈平行matrix product	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
八路遞迴matrix product	$\Theta(n^3)$	$\Theta(\lg^2 n)$	$\Theta(n^3/\lg^2 n)$
平行Strassen	$\Theta(n^{\lg 7})$	$\Theta(\lg^2 n)$	$\Theta(n^{\lg 7}/\lg^2 n)$
DOT reduction matrix product	$\Theta(n^3)$	$\Theta(\lg n)$	$\Theta(n^3/\lg n)$
Parallel merge	$\Theta(n)$	$\Theta(\lg^2 n)$	$\Theta(n/\lg^2 n)$
Parallel merge sort	$\Theta(n\lg n)$	$\Theta(\lg^3 n)$	$\Theta(n/\lg^2 n)$

表中漸近式採n充分大；空與單元素依程序基例。Work有效率、span小、空間少是不同目標；對固定P，還要把W/P、D及記憶體需求一起比較，不以一個平行度比值代替整個分析。

參考資料與署名

Cormen, Leiserson, Rivest, Stein, Introduction to Algorithms, third edition, Chapter 27. MIT 6.046J Fall 2004, Lecture 3, matrix multiplication and Strassen. MIT 6.046J Fall 2005, A Minicourse on Dynamic Multithreaded Algorithms, Handout 29.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study